

DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico

AirMiles

September 14, 2026 Algoritmos y Estructuras de Datos

Grupo 6

Integrante	LU	Correo electrónico
Gimenez, Victoria	189/25	vickygimenez002@gmail.com
Aguilera, Lola	1339/25	lolamarcolongo@gmail.com
Castro, Juan Octavio	106/18	joctavio287@gmail.com
Zarate, Eduardo Agustín	587/02	eazarate@yahoo.com.ar

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

http://www.exactas.uba.ar

TAD AirMiles {

– ***Renombres***

Distancia ES $\mathbb{R}$
FactorP ES $\mathbb{R}$
Millas ES $\mathbb{R}$
ID ES $\mathbb{Z}$
Promo ES char cadena de caracteres
Categoria ES enum {Platino, Oro, Plata, Bronce}
Transferencia ES struct $\langle origen : ID, destino : ID, millas : Millas \rangle$
Vuelo ES struct $\langle distancia : Distancia, millas : Millas, promo : Promo, factorPromocion : FactorP \rangle$ Premios no es necesario
InfoUsuario ES struct $\langle vuelos : \text{seq}\langle Vuelo \rangle, canjes : \text{seq}\langle Millas \rangle, bonos : \mathbb{Z}, premios : \mathbb{Z} \rangle$
TopeInfo ES struct $\langle tope : Millas, acumulado : Millas \rangle$

– ***Observables:*** *elegimos secuencias para vuelos, canjes y transferencias porque queremos conservar el orden temporal, a modo de historial* No es estrictamente necesario pero lo pensaron, así que la aclaración se valora :)

obs usuarios : dicc$\langle ID, InfoUsuario \rangle$
obs transferencias : seq$\langle Transferencia \rangle$
obs promociones : dicc$\langle Promo, FactorP \rangle$
obs topes : dicc$\langle Promo, TopeInfo \rangle$

– ***Predicados***

pred sinRepetidos(r : seq$\langle ID \rangle$) {
$(\forall i, j : \mathbb{Z})\ (0 \leq i < |r| \land 0 \leq j < |r| \land i \neq j \rightarrow_L r[i] \neq r[j])$
}

pred sonUsuarios(r : seq$\langle ID \rangle, s : AirMiles$) {
$(\forall i : \mathbb{Z})\ (0 \leq i < |r| \rightarrow_L r[i] \in s.usuarios)$
}

pred millasDisponiblesDeMayorAMenor(r : seq$\langle ID \rangle, s : AirMiles$) {
$(\forall i : \mathbb{Z})\ (0 \leq i < |r| - 1 \rightarrow_L$
$millasDisponibles(s, r[i]) \geq millasDisponibles(s, r[i+1]))$
}

pred sonLosDelRanking(r : seq$\langle ID \rangle, s : AirMiles$) {
$(\forall u : ID)\ (u \in s.usuarios \land u \notin r \rightarrow$
$(\forall i : \mathbb{Z})\ (0 \leq i < |r| \rightarrow_L$
$millasDisponibles(s, r[i]) \geq millasDisponibles(s, u)))$
}

pred soloCambiaUsuarios($s, S0 : AirMiles$) {
$s.transferencias = S0.transferencias \land s.promociones = S0.promociones \land s.topes = S0.topes$
}

pred soloCambiaTransferencias($s, S0 : AirMiles$) {
$s.usuarios = S0.usuarios \land s.promociones = S0.promociones \land s.topes = S0.topes$
}

pred soloCambiaPromociones($s, S0 : AirMiles$) {
$s.usuarios = S0.usuarios \land s.transferencias = S0.transferencias \land s.topes = S0.topes$

}

pred soloCambiaVuelos$(i, i' : InfoUsuario)$ {
$i.canjes = i'.canjes \land i.bonos = i'.bonos \land i.premios = i'.premios$
}

pred soloCambiaBonos$(i, i' : InfoUsuario)$ {
$i.canjes = i'.canjes \land i.vuelos = i'.vuelos \land i.premios = i'.premios$
}

– Decidimos la siguiente sintaxis aunque fuera algo reiterativa dado a la posible indefinicion, tanto para mismasClavesSalvoUna como para mismasClavesSalvoDos. Ok

pred mismasClavesSalvoUna$(d, d' : \text{dicc}\langle K, V\rangle, k' : K)$ {
$(\forall k : K)\ (k \neq k' \rightarrow ((k \in d \leftrightarrow k \in d') \land_L (k \in d \rightarrow_L d[k] = d'[k])))$
}

pred mismasClavesSalvoDos$(d, d' : \text{dicc}\langle K, V\rangle, k_1, k_2 : K)$ {
$(\forall k : K)\ ((k \neq k_1 \land k \neq k_2) \rightarrow ((k \in d \leftrightarrow k \in d') \land_L (k \in d \rightarrow_L d[k] = d'[k])))$
}

pred esSetKey$(d, d' : \text{dicc}\langle K, V\rangle, k : K, v : V)$ {
$k \in d \land mismasClavesSalvoUna(d, d', k) \land_L d[k] = v$
}

pred esDelKey$(d, d' : \text{dicc}\langle K, V\rangle, k : K)$ {
$k \notin d \land mismasClavesSalvoUna(d, d', k)$
}

pred superaOAlcanzaTope$(p : Promo, millas : Millas, s : AirMiles)$ {
$p \in s.topes \land_L (s.topes[p].acumulado + millas) \geq s.topes[p].tope$
}

pred aplicaLaPromo$(u : ID, distancia : Distancia, p : Promo, s : AirMiles)$ {
$\neg superaOAlcanzaTope(p, ganancia(s, u, distancia, p), s)$
}

pred decidePromo$(u : ID, distancia : Distancia, promo, p_f : Promo, s : AirMiles)$ {
$(aplicaLaPromo(u, distancia, promo, s) \rightarrow p_f = promo) \land$
$(\neg aplicaLaPromo(u, distancia, promo, s) \rightarrow p_f = \text{“}NoPromo\text{”})$
}

pred actualizaAcumulado$(d, d' : \text{dicc}\langle Promo, TopeInfo\rangle, promo : Promo,$
$m : Millas)$ {
$promo \in d' \land_L esSetKey(d, d', promo,$
$\langle tope : d'[promo].tope, acumulado : d'[promo].acumulado + m\rangle)$
}

pred promoPerteneceASalida$(s, S0 : AirMiles, p : Promo)$ {
$p \in s.promociones \land_L s.promociones[p] = S0.promociones[p]$
}

pred actualizaPromoYTope$(s, S0 : AirMiles, p : Promo, m : Millas)$ {
$mismasClavesSalvoUna(s.promociones, S0.promociones, p) \land$
$mismasClavesSalvoUna(s.topes, S0.topes, p) \land$
$(p \notin S0.topes \rightarrow promoPerteneceASalida(s, S0, p) \land p \notin s.topes) \land_L$
$(p \in S0.topes \rightarrow ($
$(superaOAlcanzaTope(p, m, S0) \rightarrow esDelKey(s.promociones, S0.promociones, p) \land$
$esDelKey(s.topes, S0.topes, p)) \land$

$(\neg superaOAlcanzaTope(p, m, S0) \rightarrow promoPerteneceASalida(s, S0, p) \wedge$
$actualizaAcumulado(s.topes, S0.topes, p, m))))$
}

pred seTransfirieron($u_1, u_2 : ID, s : AirMiles$) {
$(\exists i : \mathbb{Z})\ (0 \leq i < |s.transferencias| \wedge_L ($
$(s.transferencias[i].origen = u_1 \wedge s.transferencias[i].destino = u_2) \vee$
$(s.transferencias[i].origen = u_2 \wedge s.transferencias[i].destino = u_1)))$
}

pred sonExclusivos($u_1, u_2 : ID, S : AirMiles$) {
$seTransfirieron(u_1, u_2, s) \wedge$
$(\forall u : ID)\ (u \neq u_1 \wedge u \neq u_2 \rightarrow (\neg seTransfirieron(u_1, u, s) \wedge \neg seTransfirieron(u_2, u, s)))$
}

pred estanTodosLosExclusivos($r : \text{seq}\langle\text{tupla } \langle ID, ID\rangle\rangle, s : AirMiles$) {
$(\forall i : \mathbb{Z})\ (0 \leq i < |s.transferencias| \rightarrow_L ($
$sonExclusivos(s.transferencias[i].origen, s.transferencias[i].destino, s)$
$\leftrightarrow \langle s.transferencias[i].origen, s.transferencias[i].destino\rangle \in r))$
}

pred sonTodosExclusivos($r : \text{seq}\langle\text{tupla } \langle ID, ID\rangle\rangle, s : AirMiles$) {
$(\forall j : \mathbb{Z})\ (0 \leq j < |r| \rightarrow_L sonExclusivos(r[j]_0, r[j]_1, s))$
}

pred sinParesRepetidos($r : \text{seq}\langle\text{tupla } \langle ID, ID\rangle\rangle$) {
$(\forall i, j : \mathbb{Z})\ (0 \leq i < |r| \wedge 0 \leq j < |r| \wedge i \neq j \rightarrow_L ($
$\neg(r[i] = r[j] \vee_L (r[j]_0 = r[i]_1 \wedge_L r[j]_1 = r[i]_0))))$
}

– ***Procedimientos***

proc crearSistema() : $AirMiles$ {
requiere { $True$ }
asegura {
$esSetKey(res.promociones, \langle\rangle, \text{“}NoPromo\text{”}, 1.0) \wedge$
$res.usuarios = \langle\rangle \wedge res.transferencias = \langle\rangle \wedge res.topes = \langle\rangle$
} Ok
}

proc registrarUsuario(in $u : ID$, in out $s : AirMiles$) {
requiere { $s = S0 \wedge u \notin S0.usuarios$ } u >0 Los identificadores de usuario son números enteros positivos.
asegura {
$esSetKey(s.usuarios, S0.usuarios, u,$
$\langle vuelos : \langle\rangle, canjes : \langle\rangle, bonos : 0, premios : 0\rangle) \wedge$
$soloCambiaUsuarios(s, S0)$
}
}

proc registrarUsuarioConReferidor(in $u, referidor : ID$, in out $s : AirMiles$) {
requiere { $s = S0 \wedge u \notin S0.usuarios \wedge referidor \in S0.usuarios$ } u > 0
asegura {
$soloCambiaUsuarios(s, S0) \wedge$
$mismasClaveSalvoDos(s.usuarios, S0.usuarios, u, referidor) \wedge$
$u \in s.usuarios \wedge referidor \in s.usuarios \wedge_L$
$s.usuarios[u] = \langle vuelos : \langle\rangle, canjes : \langle\rangle, bonos : 1, premios : 0\rangle \wedge_L$

$s.usuarios[referidor].bonos = S0.usuarios[referidor].bonos + 1 \wedge_L$
$soloCambiaBonos(s.usuario[referidor], S0.usuario[referidor])$
}
}

proc acumularMillas(in $u : ID$, in $dis : Distancia$, in $promo : Promo$, in out $s : AirMiles$)
{
requiere {
$s = S0 \wedge u \in S0.usuarios \wedge promo \in S0.promociones$
}
dis > 0

el tope de la promoción permite que se aplique

asegura {
$s.transferencias = S0.transferencias \wedge$
$mismasClavesSalvoUna(s.usuarios, S0.usuarios, u) \wedge$
$u \in s.usuarios \wedge$
$(\exists p, m : Promo, Millas)$ (
$decidePromo(u, dis, promo, p, S0) \wedge$
$m = ganancia(S0, u, dis, p) \wedge_L$
$soloCambiaVuelos(s.usuarios[u], S0.usuarios[u]) \wedge_L$
$s.usuarios[u].vuelos = S0.usuarios[u].vuelos$
$++ \langle\langle distancia : dis, millas : m, promo : p, factorPromocion : S0.promociones[p]\rangle\rangle \wedge_L$
$actualizaPromoYTope(s, S0, p, m))$
}
} Ok

proc canjearMillas(in $u : ID$, in $millas : Millas$, in out $s : AirMiles$) {
requiere {
$s = S0 \wedge u \in S0.usuarios \wedge millas \leq millasDisponibles(S0, u) \wedge millas > 0$
}
asegura {
$mismasClavesSalvoUna(s.usuarios, S0.usuarios, u) \wedge$
$soloCambiaUsuarios(s, S0) \wedge$
$s.usuarios[u].bonos = S0.usuarios[u].bonos \wedge$
$s.usuarios[u].vuelos = S0.usuarios[u].vuelos \wedge$
$s.usuarios[u].premios = S0.usuarios[u].premios + 1 \wedge$ No es necesario
$s.usuarios[u].canjes = S0.usuarios[u].canjes ++ \langle millas \rangle$
}
}

proc transferirMillas(in $u_O, u_D : ID$, in $millas : Millas$, in out $s : AirMiles$) {
requiere {
$s = S0 \wedge u_O \in S0.usuarios \wedge u_D \in S0.usuarios \wedge u_D \neq u_O \wedge$
$millas \leq millasDisponibles(S0, u_O) \wedge millas > 0$
}
asegura {
$soloCambiaTransferencias(s, S0) \wedge$
$s.transferencias = S0.transferencias ++ \langle origen : u_O, destino : u_D, millas : m\rangle$
} Ok
}

proc consultarSaldo(in $u : ID$, in $s : AirMiles$) : $Millas$ {
requiere { $u \in s.usuarios$ }
asegura { $res = millasDisponibles(s, u)$ }
Ok

}

proc consultarCategoria(in $u : ID$, in $s : AirMiles$) : $Categoria$ {
  requiere { $u \in s.usuarios$ }
  asegura { $res = categoria(s, u)$ }
}

Ok

proc rankingMillas(in $n : \mathbb{Z}$, in $s : AirMiles$) : seq$\langle ID \rangle$ {
  requiere { $n > 0$ }
  asegura {
    $|res| = min(n, |s.usuarios|) \land sinRepetidos(res) \land sonUsuarios(res, s) \land$
    $millasDisponiblesDeMayorAMenor(res, s) \land sonLosDelRanking(res, s)$
  }
}

Ok

proc resumenTransacciones(in $u : ID$, in $s : AirMiles$) : tupla $\langle Millas, Millas, Millas \rangle$
{
  requiere { $u \in s.usuarios$ }
  asegura {
    $res = \langle millasVoladas(s, u),$
    $millasCanjeadas(s, u),$
    $(millasEnviadas(s, u) + millasRecibidas(s, u)) \rangle$
  }
}

Ok

– *Decidimos que no se puedan pisar promociones si ya existen. Y "NoPromo" se reserva únicamente para cuando se construye el TAD.*

Al no poder eliminarse NoPromo, no hay instancia en la que no pertenezca a promociones. No hace falta pedir que sea diferente, es redundante

proc iniciarPromocion(in $promo : Promo$, in $factor_p : FactorP$, in out $s : AirMiles$) {
  requiere {
    $factor_p \geq 1 \land S0 = s \land promo \neq$ "$NoPromo$" $\land promo \notin S0.promociones$
  }
  asegura {
    $s.usuarios = S0.usuarios \land s.transferencias = S0.transferencias \land$
    $esSetKey(s.promociones, S0.promociones, promo, factor_p)$
  }
}

Y no cambian los topes

proc iniciarPromocionConTope(in $promo : Promo$, in $factor_p : FactorP$,
in $t : Millas$, in out $s : AirMiles$) {
  requiere {
    $factor_p \geq 1 \land t > 0 \land S0 = s \land promo \neq$ "$NoPromo$" $\land promo \notin S0.promociones$
  }
  asegura {
    $s.usuarios = S0.usuarios \land s.transferencias = S0.transferencias \land$
    $esSetKey(s.promociones, S0.promociones, promo, factor_p) \land$
    $esSetKey(s.topes, S0.topes, promo, \langle tope : t, acumulado : 0 \rangle)$
  }
}

Ok

proc finalizarPromocion(in $promo : Promo$, in out $s : AirMiles$) {
  requiere {
    $S0 = s \land promo \neq$ "$NoPromo$" $\land promo \in S0.promociones$

}
asegura {
$s.usuarios = S0.usuarios \land s.transferencias = S0.transferencias \land$
$esDelKey(s.promociones, S0.promociones, promo) \land$
$esDelKey(s.topes, S0.topes, promo)$
} Ok
}

proc elQueTieneMasMillas(in $s : AirMiles$) : ID {
requiere { $|s.usuarios| > 0$ }
asegura {
$res \in s.usuarios \land$
$(\forall u : ID)\ (u \in s.usuarios \rightarrow_L millasTotales(s, res) \geq millasTotales(s, u))$
}
} Ok

proc elMasCanjeador(in $s : AirMiles$) : ID {
requiere { $|s.usuarios| > 0$ }
asegura {
$res \in s.usuarios \land$
$(\forall u : ID)\ (u \in s.usuarios \land u \neq res \rightarrow_L$
$(millasCanjeadas(s, res) + millasEnviadas(s, res)) \geq (millasCanjeadas(s, u) +$
$millasEnviadas(s, u)))$
}
Ok
}

proc paresTransferidoresExclusivos(in $s : AirMiles$) : seq⟨tupla $\langle ID, ID \rangle$⟩ {
requiere { $True$ }
asegura {
$estanTodosLosExclusivos(res, s) \land$
$sonTodosExclusivos(res, s) \land$
$sinParesRepetidos(res)$
}
Ok
}

– ***Auxiliares***

aux millasVoladas($s : AirMiles, u : ID$) : $Millas =$
$\sum_{i=0}^{|s.usuarios[u].vuelos|-1} s.usuarios[u].vuelos[i].millas$

aux millasCanjeadas($s : AirMiles, u : ID$) : $Millas =$
$\sum_{i=0}^{|s.usuarios[u].canjes|-1} s.usuarios[u].canjes[i]$

aux millasEnviadas($s : AirMiles, u : ID$) : $Millas =$
$\sum_{i=0}^{|s.transferencias|-1}$ IfThenElse$(s.transferencias[i].origen = u, s.transferencias[i].millas, 0)$

aux millasRecibidas($s : AirMiles, u : ID$) : $Millas =$
$\sum_{i=0}^{|s.transferencias|-1}$ IfThenElse$(s.transferencias[i].destino = u, s.transferencias[i].millas, 0)$

aux millasTotales($s : AirMiles, u : ID$) : $Millas =$
$millasVoladas(s, u) + 500 \cdot s.usuarios[u].bonos + millasRecibidas(s, u)$

aux millasDisponibles($s : AirMiles, u : ID$) : $Millas =$
$millasTotales(s, u) - millasCanjeadas(s, u) - millasEnviadas(s, u)$

aux categoria($s : AirMiles, u : ID$) : $Categoria =$

$\text{IfThenElse}(millasTotales(s,u) \geq 10^6, Platino,$
$\text{IfThenElse}(millasTotales(s,u) \geq 5 \cdot 10^4, Oro,$
$\text{IfThenElse}(millasTotales(s,u) \geq 10^4, Plata, Bronce)$
$))$

`aux factorCategoria`$(s : AirMiles, u : ID) : \mathbb{R} =$
$\text{IfThenElse}(categoria(s,u) = Platino, 0.0,$
$\text{IfThenElse}(categoria(s,u) = Oro, 2.0,$
$\text{IfThenElse}(categoria(s,u) = Plata, 1.5, 1.0)$
$))$

`aux ganancia`$(s : AirMiles, u : ID, distancia : Distancia, p : Promo) : Millas =$
$distancia \cdot factorCategoria(s,u) \cdot s.promociones[p]$

`aux min`$(n : \mathbb{Z}, m : \mathbb{Z}) : \mathbb{Z} = \text{IfThenElse}(n > m, m, n)$

}

El TP está aprobado.
Salvo las correcciones (algunos scomentario son observaciones) logra los objetivos planteados.